TRAVAUX PRATIQUES

IA pour le Génie Logiciel

Génération de tests assistée par IA pour les systèmes IA critiques

Question du cours — L'IA générative facilite la génération de tests. Est-ce suffisant quand le système testé est lui-même une IA critique ?

Master 2 Génie Logiciel · séance de 90 minutes · en binôme

78 min de TP · 7 min de débriefing · 5 min de questionnaire

Binôme : ..... / .....

Sommaire

Mode d'emploi de la séance 3
TP1 — Générer des tests avec l'IA (15 min) 5
TP2 — Critiquer les tests générés (10 min) 7
TP3 — Test de mutation (18 min) 10
TP4 — Stratégie de test d'un système autonome (25 min) 12
TP5 — Relation métamorphique (10 min) 17
Feuille de réponses 18

Mode d'emploi de la séance

1. Démarrer (moins de 5 minutes)

Choisissez **une** option. Tout est déjà installé dans l'environnement : Python, pytest, Hypothesis, outils de mutation, un LLM local (Ollama) et son modèle.

Option	Étapes
A. GitHub Codespaces (rien à installer)	Bouton **Code** → **Codespaces** → **Create codespace**. La première ouverture construit l'environnement (plusieurs minutes).
B. VS Code + Dev Containers	Ouvrez le dossier dans VS Code, puis **Reopen in Container**.
C. Docker	Docker démarré. Avec `make` (Linux, macOS) : `make build` (une seule fois, plusieurs minutes), puis `make shell`. Sans `make` (Windows) : commandes ci-dessous.

Sans `make` (Windows) : dans **PowerShell** (pas Git Bash), depuis le dossier du dépôt. Construisez l'image une seule fois. Si l'enseignant fournit une archive, utilisez plutôt `docker load -i ia4se-tests.tar`.

```
docker build -t ia4se-tests .
```

Puis, à chaque séance, ouvrez le terminal du conteneur :

```
docker run --rm -it -v "${PWD}:/workspace" -v ia4se-ollama:/opt/ollama/models -w /workspace ia4se-tests bash
```

Dans ce terminal, `make` fonctionne : toutes les commandes `make ...` du TP se tapent là.

Vérifiez que tout marche — dans le terminal du conteneur :

```
make test
```

Tous les tests doivent être verts. Sinon, appelez l'enseignant.

2. Plan de la séance

Partie	Dossier	Durée	À rendre
TP1 Générer des tests avec l'IA	`tp/tp1_generation/`	15 min	liste des cas couverts et manquants
TP2 Critiquer les tests générés	`tp/tp2_critique/`	10 min	3 tests réécrits + type de faiblesse de chaque exemple
TP3 Test de mutation	`tp/tp3_mutation/`	18 min	résumé de mutation avant/après + survivants expliqués
TP4 Stratégie de test d'un système autonome	`tp/tp4_simulator/`	25 min	tests + justification (question → technique → test) + score `make check`
TP5 Relation métamorphique	`tp/tp5_metamorphique/`	10 min	`relation.md` avec une hypothèse justifiée
Débriefing	—	7 min	—
Questionnaire	—	5 min	—

Chaque dossier contient un `INSTRUCTIONS.md` (une page) : lisez-le en premier.

3. Carte du dépôt

```
tp/
  README.md              ce fichier
  RENDU.md               vos réponses (une section par TP)
  tp1_generation/        remise.py, normalize_age.py, SPEC.md, generated_tests/
  tp2_critique/          critique_checklist.md, exemples_faibles/, test_reecrits.py
  tp3_mutation/          test_mutation.py, mutants_bonus.txt
  tp4_simulator/         sim/ (le simulateur), scenarios/, EXIGENCES.md, test_scenarios.py, check.py
  tp5_metamorphique/     model.py, transforms.py, relation.md, test_metamorphic.py
tools/                   gen_tests.py, llm.py, mini_mutate.py, random_scenarios.py
fallback/                sorties d'exemple d'un LLM (si le LLM local ne répond pas)
tests/                   tests de l'outillage du dépôt (make selftest)
```

4. Commandes

Commande	Effet
`make test` / `make test TP=4`	lance les tests (tous les TP / un seul)
`make cov` / `make cov TP=4`	couverture lignes + branches (TP1 / simulateur)
`make mutate` / `make mutate EXTRA=1`	test de mutation de `remise.py` (+ mutants bonus)
`make mutmut`	mutation avec l'outil standard mutmut (comparaison)
`make check` / `make check EQUIPE=binome-07`	score de vos tests TP4 (+ ligne pour le classement)
`python tools/voir_simulation.py jour_nominal --set luminosity=0.5`	TP4 : voir une simulation (page animée dans `tp/tp4_simulator/vues/`)
`python tools/gen_tests.py remise.py --spec SPEC.md`	génère des tests avec le LLM
`make pull-model`	télécharge le modèle LLM si l'image a été construite sans

5. Règles

- **En binôme** : l'un écrit, l'autre conteste ; échangez les rôles à chaque TP.
- **On teste, on ne corrige pas** le code fourni. Un test rouge est souvent une découverte : notez-la.
- Jugez les tests **par rapport à la spécification** (`SPEC.md`, `EXIGENCES.md`), pas par rapport au code.
- **N'ouvrez pas** `tp/tp4_simulator/.check_data/` : il contient les versions cachées utilisées par `make check`.
- LLM : le modèle local (Ollama) est utilisé par défaut. Une clé d'API personnelle peut être fournie par variable d'environnement (`OPENAI_API_KEY`, ou `ANTHROPIC_API_KEY` avec le nom du modèle dans `ANTHROPIC_MODEL`), **jamais dans le code**. LLM indisponible ? Ajoutez `--fallback` : une sortie d'exemple est copiée depuis `fallback/`.
- Bloqué ? Relisez la consigne, puis la grille ou la spécification, puis demandez à vos voisins ; après 5 minutes, appelez l'enseignant.

6. Rendre votre travail

1. Complétez `tp/RENDU.md` (TP1 à TP4) et `tp/tp5_metamorphique/relation.md` (TP5).
2. Vos tests restent dans les dossiers des TP (`test_reecrits.py`, `test_mutation.py`, `test_scenarios.py`, `test_metamorphic.py`).
3. À la fin de la séance, déposez sur Moodle une archive zip de votre dossier `tp/` (un dépôt par binôme). Sur les postes de la salle, vos fichiers peuvent être effacés à la déconnexion : déposez avant de partir.

TP1 — Générer des tests avec l'IA (15 min)

Objectif : faire générer des tests par un LLM, les exécuter, mesurer la couverture, puis établir la liste des cas couverts… et des cas oubliés, **par rapport à la spécification**.

En binôme : l'un tape les commandes, l'autre conteste (« ce test vérifie-t-il vraiment la SPEC ? »). Échangez les rôles à chaque TP.

Étapes	1	2–3	4	5–6
Durée	2 min	4 min	3 min	6 min

Étapes

1. **Lisez** `SPEC.md`, puis `remise.py` et `normalize_age.py`. Ne corrigez rien : aujourd'hui, on teste.
2. **Générez** des tests pour `remise.py` (depuis la racine du dépôt) :

   ```
   python tools/gen_tests.py remise.py --spec SPEC.md
   ```

   Résultat : `tp/tp1_generation/generated_tests/test_remise.py`. **Lisez-le en entier.** Le LLM ne répond pas (ou trop lentement) ? Utilisez la sortie d'exemple : `python tools/gen_tests.py remise.py --spec SPEC.md --fallback`
3. Recommencez avec `normalize_age.py`.
4. **Exécutez** les tests et mesurez la couverture (lignes et branches) :

   ```
   make test TP=1
   make cov
   ```

   Notez : combien de tests passent ? quelle couverture ?
5. **Comparez à la SPEC, pas au code.** Pour chaque règle de `SPEC.md`, cherchez le test qui la vérifie. Remplissez le tableau dans `tp/RENDU.md` :

Règle de la SPEC	Test qui la vérifie	Couverte ?
remise, règle 1 (≥ 10 points : −20 %)	`test_...`	oui / non / partiellement

   Pistes : les valeurs limites (4, 5, 6 et 9, 10, 11) sont-elles testées ? les valeurs négatives ? que fait le test quand le code et la SPEC ne sont pas d'accord ?
6. Mettez en regard la **couverture affichée** et le **nombre de règles réellement vérifiées**.

À rendre (dans `tp/RENDU.md`, section TP1)

La **liste des cas couverts et des cas manquants** (le tableau de l'étape 5), avec la couverture mesurée.

Si vous êtes bloqué

1. Relisez la question et la règle concernée de `SPEC.md`.
2. Comparez avec votre voisin.
3. Toujours bloqués après 5 minutes ? Appelez l'enseignant.

Défis bonus (binômes rapides)

- Affichez le prompt (`--show-prompt`), écrivez-en un meilleur dans un fichier et relancez avec `--prompt-file mon_prompt.txt`. Les tests générés sont-ils meilleurs ? Pourquoi ?
- Générez sans `--spec` : qu'est-ce qui change ?
- Lisez `tp/tp3_mutation/bonus_propriete_hypothesis.py` : une propriété Hypothesis à écrire pour `remise` (elle sert aussi au TP3).

Ce que vous devez savoir expliquer en sortant de ce TP

- Pourquoi 100 % de couverture ne garantit pas que la spécification est vérifiée.
- Ce qu'un LLM « voit » quand il génère des tests à partir du code (et ce qu'il ne voit pas).

Fichiers fournis du TP1

Spécification — module de tarification

Cette spécification fait foi. En cas de désaccord entre le code et ce document, c'est le **code** qui est à corriger.

`remise(prix, fidelite)`

Calcule le prix à payer après la remise de fidélité.

Paramètre	Signification	Domaine valide
`prix`	prix initial, en euros	nombre réel ≥ 0
`fidelite`	nombre de points de fidélité du client	entier ≥ 0

Règles :

1. **À partir de 10 points** (10 inclus) : remise de 20 % — le client paie 80 % du prix.
2. **De 5 à 9 points** (5 inclus) : remise de 10 % — le client paie 90 % du prix.
3. **Moins de 5 points** : pas de remise — le client paie le prix initial.
4. Un nombre de points **négatif** est invalide : la fonction doit lever `ValueError`.
5. Un prix **négatif** est invalide : la fonction doit lever `ValueError`.

Exemples : `remise(100, 12) == 80`, `remise(100, 7) == 90`, `remise(100, 2) == 100`.

`normalize_age(age)`

Normalise un âge saisi dans un formulaire.

1. Un âge positif ou nul est renvoyé tel quel.
2. Un âge **négatif** est invalide : la fonction doit lever `ValueError`.

Fichier `tp/tp1_generation/remise.py`

```
def remise(prix, fidelite):
    if fidelite >= 10:
        return prix * 0.80
    if fidelite >= 5:
        return prix * 0.90
    return prix
```

Fichier `tp/tp1_generation/normalize_age.py`

```
def normalize_age(age):
    return max(age, 0)
```

TP2 — Critiquer les tests générés (10 min)

Objectif : apprendre à juger un test avec une grille explicite, puis réécrire les tests faibles pour qu'ils vérifient vraiment la spécification.

En binôme : celui qui a tapé au TP1 conteste maintenant ; l'autre écrit. Pour chaque test, le « contesteur » doit convaincre l'autre avec une question de la grille.

Étapes	1	2	3	4
Durée	3 min	2 min	4 min	1 min

Étapes

1. **Classez les 3 exemples** de `exemples_faibles/` avec `critique_checklist.md` : chaque fichier a **une** faiblesse principale (code A à F). Remarquez qu'ils passent tous :

```
python -m pytest tp/tp2_critique/exemples_faibles -q
```

2. **Appliquez la grille à vos tests du TP1** (`tp/tp1_generation/generated_tests/`), en jugeant **par rapport à** `SPEC.md`, pas par rapport au code.
3. **Réécrivez au moins 3 tests** dans `test_reecrits.py`. Formes utiles :

```
@pytest.mark.parametrize("fidelite, attendu", [(4, 100), (5, 90), (6, 90)])
def test_bornes_autour_de_5(fidelite, attendu):
    assert remise(100, fidelite) == pytest.approx(attendu)   # SPEC, règles 2 et 3

def test_entree_invalide_rejetee():
    with pytest.raises(ValueError):    # forme d'un test « doit lever une exception »
        ...                            # appel avec une entrée que la SPEC déclare invalide
```

4. **Lancez-les** : `make test TP=2`. Un test réécrit échoue ? Décidez qui a tort (test ou code) et notez-le : c'est une découverte, pas une erreur.

À rendre (dans `tp/RENDU.md`, section TP2)

- Le **type de faiblesse** (A à F) de chacun des 3 exemples, justifié en une ligne.
- Vos **3 tests réécrits** (dans `test_reecrits.py`), chacun avec la règle de la SPEC qu'il vérifie.

Si vous êtes bloqué

1. Relisez la question de la grille et la règle de `SPEC.md`.
2. Comparez avec votre voisin.
3. Toujours bloqués après 5 minutes ? Appelez l'enseignant.

Défis bonus (binômes rapides)

- Écrivez la propriété Hypothesis de `tp/tp3_mutation/bonus_propriete_hypothesis.py` : elle vérifie toute une famille d'entrées d'un coup.
- Pour chaque test généré, répondez à la question 4 de la grille en imaginant le code corrigé.

Ce que vous devez savoir expliquer en sortant de ce TP

- Pourquoi un test qui passe peut être un mauvais test (il peut même protéger un bug).
- La différence entre « le test est rouge » et « le test est faux ».

Grille de critique (critique_checklist.md)

Pour chaque test, répondez **oui / non / ?** à chaque question et justifiez en une ligne. On juge le test **par rapport à la spécification** (`tp/tp1_generation/SPEC.md`), pas par rapport au code.

#	Question	Ce qu'il faut regarder
1	**Assertion forte ?**	Le test vérifie-t-il une valeur précise (`== 90`) plutôt qu'une propriété vague (`is not None`, `> 0`, `isinstance`) ?
2	**Bornes ?**	Les valeurs limites sont-elles testées : **4 / 5 / 6** et **9 / 10 / 11** ?
3	**Valeurs invalides ?**	Que demande la SPEC pour un prix ou une fidélité négatifs, un âge négatif ? Le test le vérifie-t-il (`pytest.raises(ValueError)`) ?
4	**Protège un bug ?**	Ce test **échouerait-il si le code était correct** (conforme à la SPEC) ? Si oui, il fige un bug.
5	**Redondant ?**	Apporte-t-il un cas nouveau, ou répète-t-il un autre test avec d'autres valeurs du même cas ?
6	**Couverture réelle ?**	Exécuté ≠ vérifié : la ligne est-elle seulement exécutée, ou son résultat est-il contrôlé par une assertion ?

Types de faiblesse (pour classer les exemples)

Code	Faiblesse
A	Assertion faible (question 1)
B	Bornes absentes (question 2)
C	Valeurs invalides absentes (question 3)
D	Test qui confirme un bug (question 4)
E	Test redondant (question 5)
F	Couverture sans vérification (question 6)

Un test peut cumuler plusieurs faiblesses : donnez la principale.

Les trois exemples à classer

Fichier `tp/tp2_critique/exemples_faibles/test_exemple_1.py`

```
"""Exemple 1 — tests générés pour remise()."""

from remise import remise


def test_remise_client_fidele():
    resultat = remise(100, 10)
    assert resultat is not None


def test_remise_client_regulier():
    resultat = remise(100, 5)
    assert resultat is not None
```

Fichier `tp/tp2_critique/exemples_faibles/test_exemple_2.py`

```
"""Exemple 2 — tests générés pour remise()."""

from remise import remise


def test_client_sans_points():
    assert remise(100, 0) == 100


def test_client_tres_fidele():
    assert remise(100, 50) == 80
```

Fichier `tp/tp2_critique/exemples_faibles/test_exemple_3.py`

```
"""Exemple 3 — tests générés pour normalize_age()."""

from normalize_age import normalize_age


def test_age_normal():
    assert normalize_age(42) == 42


def test_age_negatif():
    assert normalize_age(-1) == 0
```

TP3 — Test de mutation (18 min)

Objectif : mesurer l'efficacité d'une suite de tests en injectant de petites fautes (« mutants ») dans le code, puis renforcer la suite pour tuer les mutants qui survivent.

En binôme : l'un lit les survivants et propose un test, l'autre vérifie qu'il est conforme à la SPEC avant de l'ajouter. Échangez à chaque survivant.

Étapes	1	2	3–4	5	6
Durée	3 min	3 min	8 min	2 min	2 min

Le principe

Un mutant est une copie de `remise.py` avec **une** petite modification. On relance les tests :

- au moins un test échoue → le mutant est **tué** (la suite a vu la faute) ;
- tous les tests passent → le mutant **survit** (la suite est aveugle à cette faute).

Les trois mutants du cours

Mutant	Modification dans `remise.py`
M1	`fidelite >= 10` devient `fidelite > 10`
M2	`0.90` devient `0.85`
M3	`fidelite >= 5` devient `fidelite >= 6`

Question : quel test tue chacun d'eux ? Écrivez l'appel exact et la valeur attendue (par exemple `remise(100, ?) == ?`). `make mutate` les signale par « mutant du cours M1/M2/M3 ».

Étapes

1. **Lancez la mutation** sur la suite actuelle (vos tests du TP2 + `test_mutation.py`) :

   ```
   make mutate
   ```

   Copiez le **résumé** (mutants, tués, survivants) dans `tp/RENDU.md` : c'est votre « avant ». Les tests qui échouent déjà sur le code d'origine (écarts à la SPEC) sont mis de côté automatiquement : la mutation se mesure sur une suite verte.
2. **Répondez à la question des trois mutants du cours** (tableau ci-dessus).
3. Pour chaque **survivant**, lisez la ligne mutée et cherchez l'entrée qui fait différer l'original et le mutant. Exemple de forme :

   ```
   def test_tue_le_mutant_n():                # nommez le mutant visé
       assert remise(PRIX, FIDELITE) == pytest.approx(ATTENDU)   # ATTENDU d'après la SPEC
   ```

   Vérifiez à la main : avec ces valeurs, l'original et le mutant donnent-ils deux résultats différents ?
4. **Ajoutez ces tests** dans `test_mutation.py` et relancez `make mutate`.
5. Un survivant résiste ? Demandez-vous s'il est **équivalent** : aucune entrée valide de la SPEC ne distingue le mutant de l'original. Justifiez-le en une phrase.
6. Copiez le nouveau résumé : c'est votre « après ».

> Outil : `tools/mini_mutate.py` (lisible, une centaine de lignes, lancé par `make mutate`). `make mutmut` lance l'outil standard **mutmut** sur le même fichier, pour comparer (il exige une suite entièrement verte).

À rendre (dans `tp/RENDU.md`, section TP3)

- Le **résumé de mutation avant / après** (nombre de mutants, tués, survivants).
- Les **survivants expliqués** : pour chacun, le test qui l'a tué, ou la justification de son équivalence.
- La réponse pour **M1, M2, M3**.

Si vous êtes bloqué

1. Relisez la ligne mutée et la règle de `SPEC.md` concernée.
2. Comparez avec votre voisin.
3. Toujours bloqués après 5 minutes ? Appelez l'enseignant.

Défis bonus (binômes rapides)

- `make mutate EXTRA=1` ajoute les mutants écrits à la main de `mutants_bonus.txt` : lesquels sont équivalents ? lesquels révèlent une faiblesse de vos assertions ?
- Complétez la propriété Hypothesis de `bonus_propriete_hypothesis.py` : combien de mutants tue-t-elle à elle seule ?

Ce que vous devez savoir expliquer en sortant de ce TP

- Pourquoi le score de mutation dit plus que la couverture sur l'efficacité d'une suite.
- Ce qu'est un mutant équivalent, et pourquoi on ne peut pas toujours le tuer.

TP4 — Stratégie de test d'un système autonome (25 min)

Objectif : tester un système dont un composant est une IA (la perception) sans oracle parfait : choisir une **question** de sécurité, la **technique** adaptée, puis écrire le **test**. `make check` mesure combien de défauts cachés votre suite détecte, sans fausse alarme.

En binôme : l'un propose la question et la technique, l'autre écrit le test et vérifie qu'il respecte `EXIGENCES.md` (ODD, seuils). Échangez à chaque test.

Étapes	1	2	3	4	5
Durée	3 min	4 min	12 min	3 min	3 min

L'essentiel de l'API (`from sim import ...`)

```
from sim import (Scenario, run, run_baseline, load_scenario, validate_scenario, in_odd, odd_violations,
                 no_collision, min_distance_at_least, brakes_when_pedestrian_detected, check,
                 fault, CameraDegraded, GPSLost, SensorDelay, LidarNoise)

s = Scenario(car_speed=13.9, initial_distance=45, pedestrian_y=-3, pedestrian_speed=1.2,
             luminosity=0.3, occlusion=0.5)           # champs et unités : sim/scenario.py
validate_scenario(s)                                  # ValueError si physiquement impossible
r = run(s, seed=0)                                    # simulation déterministe
print(r)                                              # résumé + journal des événements
check(r, no_collision, min_distance_at_least(2.0))    # échec = message détaillé
with fault(CameraDegraded(0.6)):                      # injection de panne
    r = run(s, seed=0)
s2 = s.replace(luminosity=0.8)                        # scénario dérivé (métamorphique)
rb = run_baseline(s, seed=0)                          # système de comparaison (différentiel)
```

Créer une situation

Une situation est un `Scenario` : dix réglages. Partez d'un scénario fourni (dossier `scenarios/`) ou des valeurs par défaut, et changez ce que vous voulez avec `.replace(...)`. Le scénario de départ n'est pas modifié : `.replace` en crée un nouveau.

```
NOMINAL = load_scenario(SCENARIOS / "jour_nominal.json")          # une situation fournie
mouille = NOMINAL.replace(friction=0.5, initial_distance=35.0)    # route mouillée, passage à 35 m
gauche = Scenario(pedestrian_y=4.0, pedestrian_speed=-1.2)        # piéton venant de la gauche
```

Réglage (nom dans le code)	Signification	Défaut	Limite des exigences (ODD)
`car_speed`	vitesse de la voiture, en m/s	13.9 (50 km/h)	13,9 au plus
`initial_distance`	distance au passage piéton au départ, en m	45	voir ci-dessous
`pedestrian_y`	position du piéton en travers de la route, en m	−3	aucune
`pedestrian_speed`	vitesse du piéton en travers de la route, en m/s	1.2	de −2 à 2
`luminosity`	lumière : 0 nuit noire, 1 plein jour	1.0	0,2 au moins
`occlusion`	part du piéton masquée, de 0 à 1	0.0	0,6 au plus
`friction`	adhérence : 0,8 sec, 0,5 mouillé, 0,1 verglas	0.8	0,4 au moins
`sensor_noise`	imprécision du lidar, en m	0.1	0,3 au plus
`sensor_latency_ms`	retard normal des mesures, en ms	0	100 au plus
`sensor_dropout`	part des trames capteur perdues, de 0 à 1	0.0	0,3 au plus

- `initial_distance` : au moins la distance de freinage + 15 m, soit 27,3 m sur route sèche à 50 km/h, davantage sur route mouillée. `in_odd` fait le calcul pour vous.
- `pedestrian_y` : 0 est le milieu de la voie de la voiture, qui va de −1,75 à +1,75 ; une valeur négative est sur le trottoir de droite, où le piéton attend par défaut.

- `pedestrian_speed` : positive, le piéton va vers les y croissants (depuis le trottoir de droite, il traverse) ; négative, il va dans l'autre sens ; 0, il reste immobile.
- `sensor_latency_ms` : au-delà de 100 ms, ce n'est plus un réglage mais une panne (`SensorDelay`).

Les pannes se placent autour de `run`, avec `with fault(...)` :

Panne	Signification	Limite de R4
`CameraDegraded(level)`	caméra salie ou éblouie : 0 intacte, 1 aveugle	tout niveau
`SensorDelay(ms)`	retard supplémentaire de toutes les mesures, en ms	300 au plus
`LidarNoise(sigma)`	bruit supplémentaire du lidar, en m	0,3 au plus
`GPSLost()`	plus de signal GPS	toujours

R4 couvre **une** panne à la fois. **La graine** (`run(s, seed=3)`) fixe le hasard des capteurs : même situation et même graine donnent le même résultat. Bouclez sur plusieurs graines pour tester une situation plusieurs fois.

Rester dans les limites (sinon : fausse alarme)

Les exigences R1 à R3 ne s'appliquent que **dans l'ODD** (colonne de droite du premier tableau), et R4 seulement avec **une** panne dans ses limites. En dehors, la voiture n'a rien promis : un test qui y exige « pas de collision » peut échouer aussi sur la version correcte. C'est une **fausse alarme**, et votre score de `make check` tombe à 0.

Mettez la vérification dans le test, avant d'exiger un bon comportement :

```
def test_route_mouillee():
    scenario = NOMINAL.replace(friction=0.5, initial_distance=35.0)
    assert in_odd(scenario), odd_violations(scenario)    # s'arrête ici, avec la raison, si hors limites
    check(run(scenario, seed=0), no_collision)
```

`odd_violations(scenario)` explique ce qui sort des limites. Pour une route verglacée (`friction=0.1`), elle répond : `friction = 0.1 hors de l'ODD [0.4, 1.0]` et `initial_distance = 45.0 < 113.5 m nécessaires`. Une situation physiquement impossible (une latence de 300 ms comme réglage, par exemple) est refusée dès `run`, avec un message.

Voir une simulation

Depuis la racine du dépôt, dans le terminal du conteneur :

```
python tools/voir_simulation.py jour_nominal                                    # un scénario fourni
python tools/voir_simulation.py jour_nominal --set friction=0.5 initial_distance=35   # avec vos réglages
python tools/voir_simulation.py jour_nominal --panne "LidarNoise(0.2)" --graine 3     # avec une panne
python tools/voir_simulation.py jour_nominal --comparer friction=0.5            # deux voitures côte à côte
python tools/voir_simulation.py jour_nominal --baseline                         # avec le robot simple
```

La commande affiche le résumé de la simulation et écrit une page animée dans `tp/tp4_simulator/vues/` : vue de dessus, vitesse, distance au piéton, journal. Ouvrez-la sur votre ordinateur par un double-clic (Codespaces : clic droit sur le fichier, puis *Download*). La page signale **HORS DES LIMITES** toute situation où les exigences ne s'appliquent pas.

Depuis un test, `voir` écrit la page des résultats que vous lui donnez :

```
from tools.voir_simulation import voir

result = run(scenario, seed=0)
voir(result)                 # ou voir(result_1, result_2) pour comparer deux exécutions
```

Étapes

1. **Lisez** `EXIGENCES.md` (exigences R1–R5, domaine opérationnel) et lancez `make test TP=4` : les deux tests fournis passent.
2. **Le cas du cours** : nuit + piéton partiellement caché (`scenarios/nuit_pieton_partiellement_cache.json`). Écrivez le test dans `test_scenarios.py`, lancez-le, lisez `print(result)`. Que se passe-t-il ? Pour le voir : `python tools/voir_simulation.py nuit_pieton_partiellement_cache`.
3. **Une question, une technique, un test.** Écrivez au moins un test par technique :
   - **Génération de scénarios** : valeurs limites de l'ODD, familles de scénarios, tirage aléatoire (`python tools/random_scenarios.py --n 200` : que contient vraiment un tirage naïf ?). Bouclez sur plusieurs scénarios et plusieurs graines.
   - **Injection de pannes** : `with fault(...)` (bornes de R4).
   - **Métamorphique** : une transformation du scénario qui ne doit pas rendre le système moins sûr (exemple : plus de lumière, toutes choses égales par ailleurs).
   - **Différentiel** : comparez `run` et `run_baseline` (freinage d'urgence simple, sans IA) : quand la baseline évite le piéton, le système complet doit l'éviter aussi.

   Choisissez l'oracle le plus fort justifié par EXIGENCES.md (pas seulement `no_collision`). Option LLM : `python tools/random_scenarios.py --llm "piétons difficiles à voir"` demande des scénarios au LLM puis les passe à `validate_scenario()` (sans LLM : `--llm-fallback`).
4. **Évaluez** votre suite : `make check`. Une fausse alarme (échec sur la version correcte) annule le score : corrigez l'oracle ou sortez le scénario de vos tests s'il est hors ODD.
5. **Justifiez** dans `tp/RENDU.md` : tableau question → technique → test, et score de `make check`.

> Règle du jeu : le dossier `.check_data/` contient les versions cachées utilisées par `make check`. **Ne l'ouvrez pas et ne cherchez pas à le décoder** : le but est de trouver les défauts par vos tests. Lire le code de `sim/` est en revanche autorisé.

À rendre (dans `tp/RENDU.md`, section TP4)

- Vos **tests** (`test_scenarios.py`, et d'autres fichiers `test_*.py` si besoin).
- La **justification** de chaque test : question → technique → test.
- Le **score** affiché par `make check` (et les variantes détectées).

Si vous êtes bloqué

1. Relisez la question, `EXIGENCES.md` et le journal (`print(result)`).
2. Comparez avec votre voisin.
3. Toujours bloqués après 5 minutes ? Appelez l'enseignant.

Défis bonus (binômes rapides)

- **Famille de scénarios « pluie »** : `python tools/random_scenarios.py --famille pluie --n 30` (fonction `famille_pluie()` dans `tools/random_scenarios.py`). Écrivez le test qui l'utilise. Trouve-t-il un défaut que vos autres tests ne trouvent pas ?
- Écrivez une propriété Hypothesis sur des scénarios tirés dans l'ODD (`@given` avec `st.floats(...)` pour chaque champ, puis `assume(in_odd(s))`).
- Testez R5 (mode dégradé GPS) à partir de `result.trace`.

Ce que vous devez savoir expliquer en sortant de ce TP

- Pourquoi la couverture ne suffit pas pour un système dont la perception est une IA.
- Quelle technique a trouvé quel défaut, et pourquoi les autres ne pouvaient pas le trouver.
- Pourquoi une fausse alarme compte autant qu'un défaut manqué.

Exigences du système (EXIGENCES.md)

Système testé : un véhicule autonome **jouet** approche d'un passage piéton. Chaîne de traitement : capteurs → perception (classe + confiance) → prédiction (suivi, trajectoire) → planification (continuer / ralentir / freiner) → contrôle (décélération). Modèle simplifié à visée pédagogique : il ne reproduit aucun véhicule ni aucun événement réel.

Domaine opérationnel (ODD)

Les exigences R1 à R3 s'appliquent aux scénarios **valides** (`validate_scenario`) et **dans l'ODD** (`in_odd(scenario)` ; `odd_violations(scenario)` explique pourquoi un scénario en sort).

Paramètre	Dans l'ODD
`car_speed`	≤ 13,9 m/s (50 km/h)
`friction`	≥ 0,4
`luminosity`	≥ 0,2
`occlusion`	≤ 0,6
`sensor_noise`	≤ 0,3 m
`sensor_dropout`	≤ 0,3
`pedestrian_speed`	entre −2 et 2 m/s
`sensor_latency_ms`	≤ 100 ms (toujours : c'est la spécification du capteur)
`initial_distance`	≥ distance de freinage (`stopping_distance(car_speed, friction)`) + 15 m

Exigences

Id	Exigence	Oracle fourni
R1	Le véhicule ne percute jamais le piéton.	`no_collision`
R2	Tant que le piéton est sur la voie du véhicule, la distance entre eux reste **d'au moins 2 m**.	`min_distance_at_least(2.0)`
R3	Si les capteurs mesurent le piéton alors qu'il est sur la voie devant le véhicule, le véhicule **freine dans la seconde** (ou est déjà arrêté).	`brakes_when_pedestrian_detected`
R4	Avec **une** panne parmi `CameraDegraded` (tout niveau), `SensorDelay` (jusqu'à 300 ms), `LidarNoise` (σ ≤ 0,3 m), `GPSLost`, R1 reste vraie dans l'ODD.	`with fault(...)` + `no_collision`
R5	En cas de perte GPS, le véhicule passe en mode dégradé et ralentit vers 30 km/h (8,3 m/s) s'il n'y a pas d'obstacle.	à écrire (`result.trace`, `result.final_speed`)

En dehors de l'ODD ou au-delà des pannes de R4, **rien n'est exigé** : un test qui y attend une absence de collision est un oracle injustifié (il peut échouer sur un système correct).

Ce que renvoie `run(scenario, seed)`

`Result` : `collision` (bool), `collision_time`, `min_distance` (m, mesurée tant que le piéton est sur la voie), `first_detection_time` (s), `final_speed`, `log` (journal lisible), `trace` (un dictionnaire par pas de 50 ms : `t`, `car_x`, `speed`, `pedestrian_y`, `observed`, `action`, `decel`). `print(result)` affiche un résumé et le journal. Même scénario + même graine (`seed`) = même résultat.

TP5 — Relation métamorphique (10 min)

Objectif : tester un modèle d'IA **sans oracle** (on ne connaît pas toujours la bonne réponse) grâce à une relation métamorphique, et apprendre à justifier cette relation.

En binôme : l'un écrit la relation (5 champs), l'autre joue l'avocat du diable sur l'**hypothèse** avant qu'on écrive le moindre code. Puis échangez pour le test.

Étapes	1	2	3	4
Durée	2 min	3 min	3 min	2 min

Le système testé : un classifieur de chiffres manuscrits (images 8x8) entraîné sur le jeu « digits » de scikit-learn (`model.py`). Transformations disponibles : `transforms.py` (`brightness`, `contrast`, `noise`, `translate`, `rotate`, `fog`).

```
import model
from transforms import rotate
images, labels = model.test_images()          # ou model.test_images(digit=6)
label, confidence = model.predict(images[0])
avant = model.predict_many(images)
apres = model.predict_many([rotate(img, 10) for img in images])
```

Étapes

1. **Lisez et lancez** la relation fournie : `make test TP=5` (`test_metamorphic.py`). Elle est vérifiée. Que devient-elle si vous remplacez `0.03` par `0.2` ? La tolérance était-elle justifiée ?
2. **Remplissez** `relation.md` pour une relation **à vous** : entrée, transformation, sortie attendue, **hypothèse justifiée**, critère de violation. Essayez une transformation géométrique (translation, rotation) ou une dégradation (brouillard, bruit).
3. **Écrivez le test** `test_relation_2_...` dans `test_metamorphic.py` et lancez-le.
4. **Si la relation est violée**, regardez les images concernées (`print(np.round(img, 1))`) et tranchez : **bug du modèle** (l'hypothèse tient, le modèle a tort) ou **relation injustifiée** (l'hypothèse ne tient pas pour ces entrées) ?

À rendre

`tp/tp5_metamorphique/relation.md` complété : la relation (5 champs) **avec une hypothèse justifiée**, le résultat du test et votre verdict « bug du modèle ou relation injustifiée ».

Si vous êtes bloqué

1. Relisez la relation fournie et les docstrings de `transforms.py`.
2. Comparez avec votre voisin.
3. Toujours bloqués après 5 minutes ? Appelez l'enseignant.

Défis bonus (binômes rapides)

- Une **seconde relation**, d'une autre famille, avec son hypothèse : `relation_bonus.md`.
- Trouvez l'intensité à partir de laquelle votre relation cesse d'être vérifiée (balayage du paramètre).

Ce que vous devez savoir expliquer en sortant de ce TP

- Ce qu'une relation métamorphique remplace (l'oracle) et ce qu'elle ne remplace pas.
- Pourquoi une violation n'est pas automatiquement un bug du modèle.

Feuille de réponses

Même contenu que `tp/RENDU.md` et `tp/tp5_metamorphique/relation.md` : remplissez les fichiers, ou cette feuille si l'enseignant le demande.

TP1 — Cas couverts et manquants

Couverture mesurée (`make cov`) : lignes … % ; branches … %. Tests générés : … ; verts :

Règle de la SPEC	Test qui la vérifie	Couverte ? (oui / non / partiellement)
remise, règle 1 (≥ 10 points : −20 %)		
remise, règle 2 (5 à 9 points : −10 %)		
remise, règle 3 (< 5 points : pas de remise)		
remise, règle 4 (fidélité négative : ValueError)		
remise, règle 5 (prix négatif : ValueError)		
normalize_age, règle 1 (âge ≥ 0 inchangé)		
normalize_age, règle 2 (âge négatif : ValueError)		

Cas manquants importants :

TP2 — Critique

Exemple	Type de faiblesse (A à F)	Justification (une ligne)
`exemples_faibles/test_exemple_1.py`		
`exemples_faibles/test_exemple_2.py`		
`exemples_faibles/test_exemple_3.py`		

Tests réécrits (dans `tp2_critique/test_reecrits.py`) : … (pour chacun : règle vérifiée, et résultat — vert, ou rouge parce que le code ne respecte pas la SPEC ?)

TP3 — Mutation

	Mutants	Tués	Survivants	Score
Avant				
Après				

Mutants du cours : M1 tué par … ; M2 tué par … ; M3 tué par …

Survivants restants et explication (test ajouté, ou pourquoi le mutant est équivalent) :

TP4 — Stratégie de test

Question de sécurité	Technique (scénarios, pannes, métamorphique, différentiel)	Test (nom)	Exigence

Score `make check` : … / 5 ; fausse alarme : oui / non ; variantes détectées :

Quelle technique a trouvé quoi ? Qu'est-ce qu'aucun de vos tests n'aurait pu trouver ?

TP5

Voir `tp/tp5_metamorphique/relation.md`.

TP5 — Votre relation métamorphique

Binôme : .....

Remplissez les **cinq champs** avant d'écrire le test. **Une relation sans hypothèse justifiée est incomplète.**

1. Entrée (*Input*)

Quelles images ? (toutes les images de test, une classe précise, celles prédites avec une confiance ≥ … ?)

2. Transformation

Fonction de `transforms.py` et valeur du paramètre d'intensité.

3. Sortie attendue (*Expected output*)

Relation entre la sortie avant et après transformation (même classe ? confiance qui ne baisse pas de plus de … ?).

4. Hypothèse (*Hypothesis*) — obligatoire

Pourquoi cette transformation **ne devrait pas** changer la réponse ? Sur quoi repose cette affirmation (perception humaine, physique du capteur, définition de la classe) ? Pour quelle plage d'intensité reste-t-elle vraie ?

5. Critère de violation (*Violation criterion*)

À partir de quand dit-on que la relation est violée (une image ? plus de x % ?). Justifiez le seuil.

Résultat

- Test : `test_metamorphic.py::test_relation_2_…`
- Relation **vérifiée** ou **violée** ? Sur combien d'images ?
- Si elle est violée : **bug du modèle ou relation injustifiée ?** Argumentez en 2–3 lignes (regardez quelques images concernées : un humain dirait-il la même chose que la relation ?).